Университет ИТМО

Факультет Программной Инженерии и Компьютерной техники

Программирование

Лабораторная работа №3

Вариант № 1452

Выполнил:

Крахмальников Илья Артемович

Группа P3119

Преподаватели:

Бойко В. А.

Гаврилов А. В.

Санкт-Петербург

2026

Содержание

Задание 3
Основные этапы решения 4
Анализ задания 4
Сценарий 6
Побег повара 6
Война 6
Праздник 6
Диаграмма классов 7
Реализация 8
Структура проекта 8
Реализация классов и интерфейсов 8
Реализация метода startBattle 9
Исходный код 12
Выводы по работе 13

Задание

Введите вариант:

1452

Описание предметной области, по которой должна быть построена объектная модель:

> Князь же утверждал, что герцог сам дал ему возможность тайком убе- жать, чтобы не потерять своего лучшего повара, и упрекал его в том, что он нарушил свое слово. Из-за этого между обоими государями возникла продолжительная война, которая известна историкам названием "Травяной войны". Обе стороны дали несколько сражений, но в конце концов был заключен мир, который получил название "Паштетного", так как на празднике в честь примирения повар князя подал к столу паштет-сюзерен, которому герцог и оказал должную честь.

В соответствии с выданным вариантом на основе предложенного текстового отрывка из литературного произведения создать объектную модель реального или воображаемого мира, описываемого данным текстом. Должны быть выделены основные персонажи и предметы со свойственным им состоянием и поведением. На основе модели написать программу на языке Java.

Этапы выполнения работы:

1. Получить вариант
2. Нарисовать UML-диаграмму, представляющую классы и интерфейсы объектной модели и их взаимосвязи;
3. Придумать сценарий, содержащий действия персонажей, аналогичные приведенным в исходном тексте;
4. Согласовать диаграмму классов и сценарий с преподавателем.
5. Написать программу на языке Java, реализующую разработанные объектную модель и сценарий взаимодействия и изменения состояния объектов. При запуске программа должна проигрывать сценарий и выводить в стандартный вывод текст, отражающий изменение состояния объектов, приблизительно напоминающий исходный текст полученного отрывка.
6. Для одного из методов класса персонажа (или объекта), согласованного с преподавателем, создать промпты для 2-3 ИИ-ассистентов (Gigachat, DeepSeek, Qwen, или др.), получить от них код метода, **реализовать метод самостоятельно**, и включить в отчет сравнительный анализ кода, написанного ИИ-ассистентами, и своего варианта. Оценить качество кода, полученного от ИИ-ассистентов, указать на возможные недоработки.
7. Продемонстрировать выполнение программы на сервере `helios`.
8. Ответить на контрольные вопросы и выполнить дополнительное задание.

Текст, выводящийся в результате выполнения программы не обязан дословно повторять текст, полученный в исходном задании. Также не обязательно реализовывать грамматическое согласование форм и падежей слов выводимого текста.

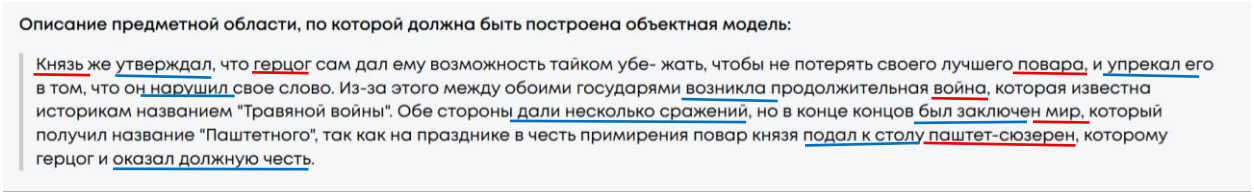
Стоит отметить, что цель разработки объектной модели состоит не в выводе текста, а в эмуляции объектов предметной области, а именно их состояния (поля) и поведения (методы). Методы в разработанных классах должны изменять состояние объектов, а выводимый текст должен являться побочным эффектом, отражающим эти изменения.

Требования к объектной модели, сценарию и программе:

1. В модели должны быть представлены основные персонажи и предметы, описанные в исходном тексте. Они должны иметь необходимые атрибуты и характеристики (состояние) и уметь выполнять свойственные им действия (поведение), а также должны образовывать корректную иерархию наследования классов.
2. Объектная модель должна реализовывать основные принципе ООП - инкапсуляцию, наследование и полиморфизм. Модель должна соответствовать принципам SOLID, быть расширяемой без глобального изменения структуры модели.
3. Сценарий должен быть вариативным, то есть при изменении начальных характеристик персонажей, предметов или окружающей среды, их действия могут изменяться и отклоняться от базового сценария, приведенного в исходном тексте. Кроме того, сценарий должен поддерживать элементы случайности (при генерации персонажей, при задании исходного состояния, при выполнении методов).
4. Объектная модель должна содержать как минимум один корректно использованный элемент каждого типа из списка:
   - абстрактный класс как минимум с одним абстрактным методом;
   - интерфейс;
   - перечисление (enum);
   - запись (record);
   - массив или ArrayList для хранения однотипных объектов;
   - проверяемое исключение.
5. В созданных классах основных персонажей и предметов должны быть корректно переопределены методы `equals()`, `hashCode()` и `toString()`. Для классов-исключений необходимо переопределить метод `getMessage()`.
6. Созданные в программе классы-исключения должны быть использованы и обработаны. Кроме того, должно быть использовано и обработано хотя бы одно unchecked исключение (можно свое, можно из стандартной библиотеки).
7. При необходимости можно добавить внутренние, локальные и анонимные классы.

Основные этапы решения

Анализ задания



Описание предметной области, по которой должна быть построена объектная модель:

> Князь же утверждал, что герцог сам дал ему возможность тайком убе- жать, чтобы не потерять своего лучшего повара, и упрекал его в том, что он нарушил свое слово. Из-за этого между обоими государями возникла продолжительная война, которая известна историкам названием "Травяной войны". Обе стороны дали несколько сражений, но в конце концов был заключен мир, который получил название "Паштетного", так как на празднике в честь примирения повар князя подал к столу паштет-сюзерен, которому герцог и оказал должную честь.

У нас 6 объектов:

- Князь
- Герцог
- Повар
- Война
- Мир
- Паштет-сюзерен (Блюдо)

Князь и Герцог – являются лидерами (Leader) (то есть наследуются от общего абстрактного класса), которые могут:

- Объявлять войну
- Заключить мир

Князь (Prince) может:

- Утверждать
- Упрекать

Герцог (Duke) может:

- Дать возможность сбежать (giveEscape)
- Оказать должную честь

Повар (Cook) может:

- Тайком убежать (escape)
- Подать к столу еду (serveDish)

Князь, Герцог и Повар являются людьми, поэтому все 3 наследуются от класса Person, у которого определено поле name, а также методы hashCode(), equals(), toString()

Мир (Contract) – объект, который ничего не может делать, лишь имеет название и создается при окончании войны, можно его задать через record

Война (War) – объект, который хранит информацию о войне между двумя лидерами, а также возможно:

- Провести сражение
- Завершить войну

Блюдо (Cook) – объект, который используется для готовки еды, так Паштет-сюзерен является одним из блюд, будет enum:

- Паштет-сюзерен
- Яблочный пирог
- Пицца пеперони

Для рандомного сценария сделаем рандомный шанс разрешить сбежать повару. Если не дали возможность сбежать повару, то он не может сбежать и выкидываем проверяемую ошибку CantEscape

Для абстрактного метода у Person реакции добавим метод reactTo, который позволяет проявлять реакцию пользователя

Если война закончилась, но никто из лидеров не закончилось терпение, то война завершается ничьей

Если войну попробует закончить лидер, кто не является ее участником, то выкинет ошибку

Сценарий

Побег повара

1. Создаются объекты: Герцог, Князь и Лучший повар
2. Далее у Герцога вызывается метод, внутри него генерируется случайное число. Если оно больше 0.3, Герцог дает возможность повару сбежать, иначе разрешения не будет
3. Повар пытается вызывает метод побега

- Если Герцог не разрешил, выбрасывается проверяемое исключение CantEscape, программа ловит его, выводит сообщение об ошибке, и войны не будет
- Если разрешил, повар успешно сбегает, и программа переходит дальше

Война

1. Князя утверждает, что Герцог дал сбежать повару, и упрекает его в нарушении слова
2. Князь начинает войну. Создается объект War, статус которого устанавливается в WAR. Война добавляется в списки активных войн обоих лидеров
3. Проводятся три сражения:

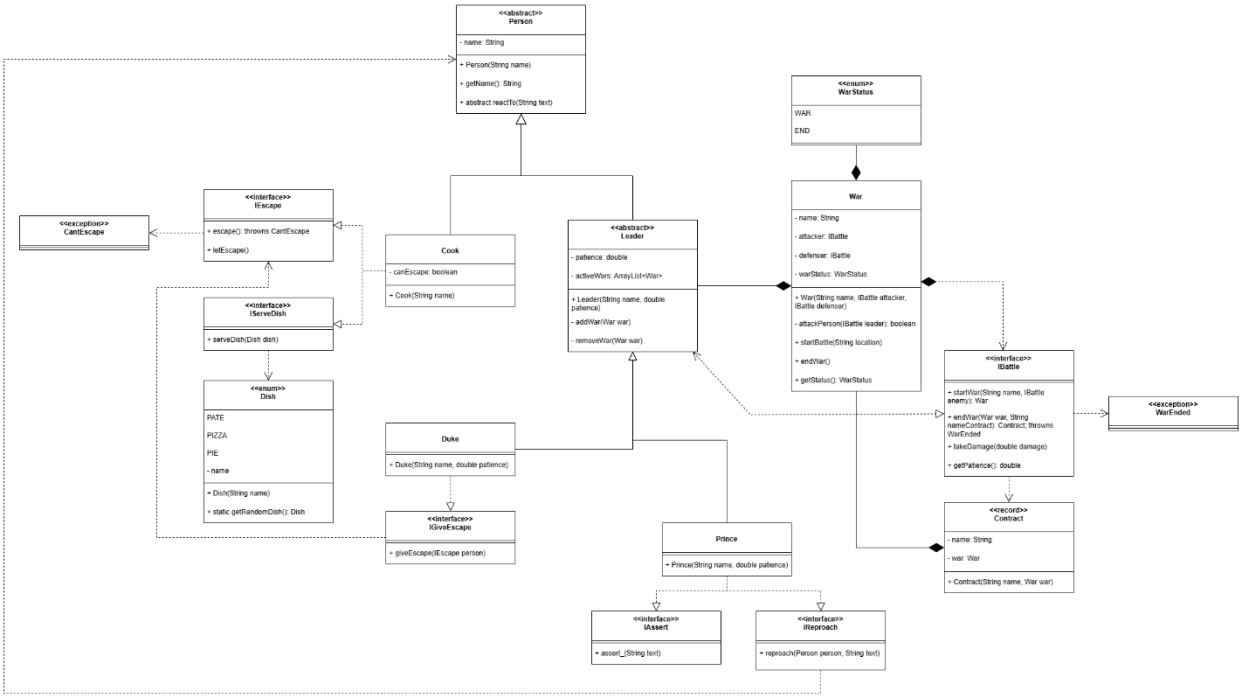
- Случайным образом генерируется урон (от 0 до 30), который уменьшает терпение одного из лидеров
- Если терпение одного из них падает до 0, сражения останавливаются. Создается объект record Contract с названием мира. Статус войны меняется на END, а война удаляется из списков активных.
- Если терпение лидера не достигло нуля за 3 битвы, война формально продолжается, но сценарий переходит к празднику

4. Если после окончания войны будет предпринята попытка провести еще одно сражение, программа выбросит и поймает непроверяемое исключение WarEnded

Праздник

1. В случае, если война закончилась, то начинается праздник
2. Повар подает на стол блюдо
3. Создается анонимный класс Person (Иностранный гость), который удивляется чести, оказанной блюду
4. Герцог также реагирует на событие

Диаграмма классов



https://github.com/124476/itmo/blob/master/programming/lab3/docs/class_diagram.drawio.pdf

Реализация

Структура проекта

Проект разделен на 4 папки: enum, exception, interface, model, а также имеет отдельный файл Main.java для запуска. Структура проекта представлена на Рисунок 1 – структура проектаРисунок 1

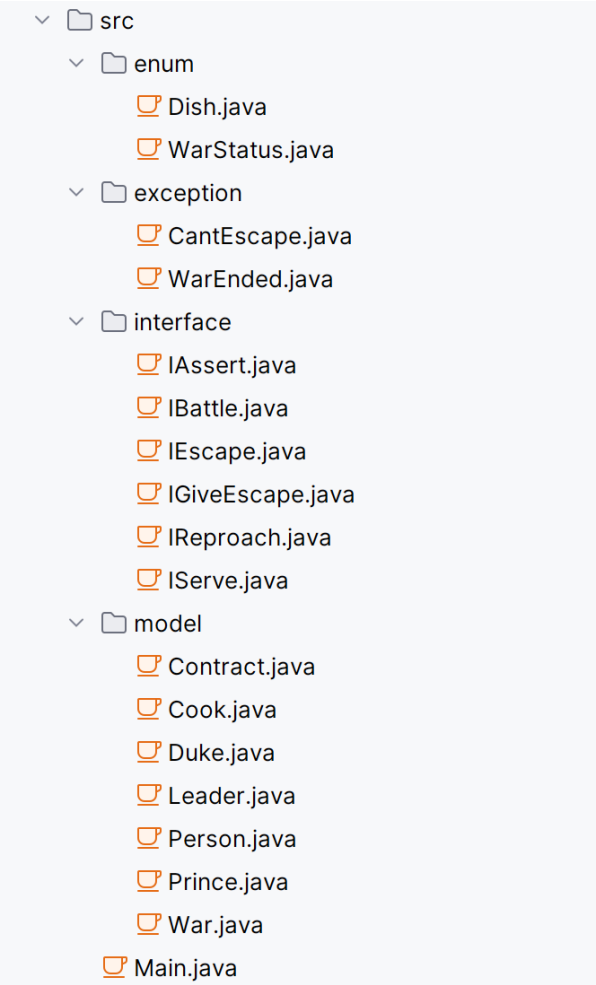


Рисунок 1 – структура проекта

Реализация классов и интерфейсов

Базовые сущности

- Person (абстрактный класс) - базовый класс для всех персонажей, содержит имя и абстрактный метод reactTo
- Leader (абстрактный класс) - наследуется от Person и реализует IBattle. Содержит параметр patience и activeWars
- Prince и Duke - конкретные реализаторы Leader
- Cook - наследуется от Person и реализует интерфейсы IEscape и IServeDish

Интерфейсы поведения

- IBattle — определяет контракт для ведения войны (startWar, endWar, takeDamage, getPatience)
- IEscape — позволяет персонажу сбегать (escape, letEscape)
- IServeDish — позволяет подавать блюда (serveDish)
- IReproach — позволяет выражать упреки (reproach)
- IGiveEscape — позволяет предоставить возможность побега (giveEscape)
- IAssert — позволяет выражать претензии (assert_)

Вспомогательные элементы

- War — класс, описывающий сам конфликт
- Contract — запись (record), описывающая мирный договор (содержит имя и ссылку на войну)
- WarStatus (enum) — статусы войны (WAR, END)
- Dish (enum) — перечисление блюд (PATE, PIZZA, PIE)
- Исключения WarEnded и CantEscape для контроля состояний

Реализация метода startBattle

Deepseek предложил код, представленный на Рисунок 2

```java
public void startBattle(String location) {  no usages  124476 *
    if (this.warStatus == WarStatus.END) {
        System.out.println("Война уже завершена. Битва невозможна.");
        return;
    }

    System.out.println("Начало битвы в локации: " + location);
    System.out.println("Нападающий: " + attacker.getClass().getSimpleName());
    System.out.println("Защитник: " + defenser.getClass().getSimpleName());

    boolean isDefenderAlive = true;
    boolean isAttackerAlive = true;

    // Пока обе стороны сохраняют терпение и война не окончена
    while (isAttackerAlive && isDefenderAlive && this.warStatus == WarStatus.WAR) {

        // Нападающий атакует защитника
        isDefenderAlive = attackPerson(this.defenser);
        if (!isDefenderAlive) {
            System.out.println("Защитник потерял терпение! Он терпит поражение.");
            endWar(); // Завершаем войну
            break;
        }

        // Защитник наносит ответный удар
        isAttackerAlive = attackPerson(this.attacker);
        if (!isAttackerAlive) {
            System.out.println("Нападающий потерял терпение! Он вынужден отступить.");
            endWar(); // Завершаем войну
            break;
        }
```

Рисунок 2 – код, предложенный deepseek

GigaChat предложил следующий код (представлен на Рисунок 3)

```java
public void startBattle(String location) {  no usages  124476 *
    if (warStatus == WarStatus.END) {
        throw new WarEnded(this.name);
    }

    logger.log("Произошло сражение: " + location);

    if (defenser == null) {
        logger.log("Ошибка: Защищающаяся сторона не определена. Сражение отменено.");
        return;
    }

    if (attackPerson(defenser)) {
        endWar();
        return;
    }

    if (attacker == null) {
        logger.log("Ошибка: Атакующая сторона не определена. Сражение прервано.");
        return;
    }

    if (warStatus == WarStatus.END) {
        return;
    }

    attackPerson(attacker);
}
```

Рисунок 3 – код, предложенный GigaChat

Моя реализация метода представлена на Рисунок 4

```
public void startBattle(String location) {  no usages   124476
    if (warStatus == WarStatus.END) {
        throw new WarEnded(this.name);
    }

    logger.log("Произошло сражение: " + location);

    if (attackPerson(defenser)) return;
    attackPerson(attacker);
}
```

Рисунок 4 – собственная реализация метода startBattle

Сравнении реализации метода ИИ-ассистентами и мной:

Критерий	Deepseek	GigaChat	Собственная
Симуляция боя	Цикл while – до победы	Один раунд	Один раунд
Обработка окончания войны	return	throw new WarEnded	throw new WarEnded
Проверка на null	Нет	Есть	Нет
Читаемость кода	Средняя (много флагов)	Низкая (много if и пробелов)	Высокая

Недостатки ИИ-ассистентов:

Deepseek проигнорировал исключение new WarEnded, а также использовал для логирования System.out.println

GigaChat добавил проверки на null, что является избыточным. Код if (warStatus == WarStatus.END) return – после attackPerson(defenser) является мертвым, поскольку если война закончится, то она уже автоматически завершится

Вывод: Собственная реализация является более читаемой и не имеющая лишний функционал, по сравнению с реализациями ИИ-ассистентов.

Исходный код

https://github.com/124476/itmo/tree/master/programming/lab3

Выводы по работе

1. Изучил принципы SOLID и STUPID
2. Научился работать с интерфейсами, использовать их для реализации методов объектов
3. Научился использовать ИИ-ассистентов для реализации кода, при этом понял, что они не способны на данный момент самостоятельно написать максимально лаконичный код